

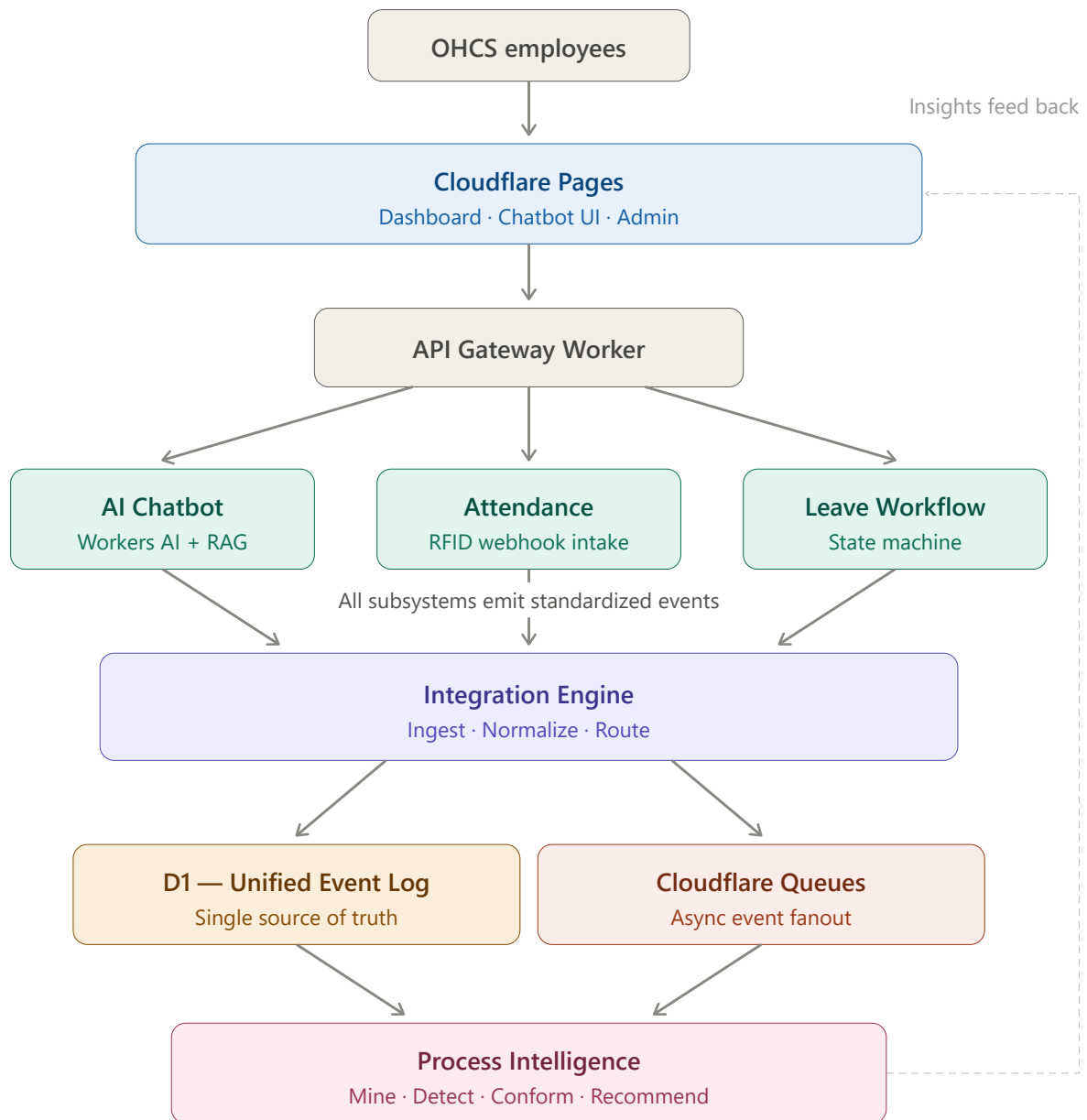
IIE Demonstration Guide

For: Student demonstrating the Intelligent Integration Engine (IIE) to a project supervisor **System:** OHCS Process Intelligence Platform — Cloudflare Workers + D1 + Workers AI + Vectorize, React dashboard **Time needed:** 20–30 minutes for the full walkthrough

1. The 30-second pitch (memorise this)

"IIE is a process intelligence platform for the Office of the Head of Civil Service. It collects a unified event log from three source systems — RFID attendance, the leave workflow, and an HR chatbot — and automatically mines that log to discover how processes actually run: it draws the real process map, finds bottlenecks, flags cases that skip required approval steps, and turns all of that into decision-support recommendations for management. Staff get self-service leave with the real OHCS approval chains built in — standard leave through F&A, study leave through RTDD — and an AI assistant that answers HR policy questions and can submit leave requests in plain language."

Key phrase if asked what makes it different from a normal dashboard: **"A normal dashboard shows you numbers you asked for. Process mining discovers the actual workflow from the event log and tells you where reality deviates from policy — without anyone configuring it."**



System architecture — source subsystems publish canonical events into the unified log; mining and the dashboard consume it.

2. Pre-demo checklist (do this 30 minutes before)

Option A — present from the live site (recommended; no localhost, no terminal)

The platform is deployed at <https://iie.ghwmelite.workers.dev>, already seeded with the full demo dataset. To prepare:

1. Open <https://iie.ghwmelite.workers.dev> in your browser.
2. Click through all four tabs once to warm everything up, then click **Presenter** in the header and step forward/back a couple of steps to get comfortable. Keep this window ready.

That's it — no dev server, no seeding, no terminal. The whole presentation (pitch + Acts 1–5) runs in the browser against the live URL. Only the *optional* Act 6 validation harness needs a terminal (see §9).

Notes for the live site:

- It works from any machine with internet — you can present from the supervisor's own computer if needed.
- Rehearsing on the live site adds a few real leave cases and chatbot events to the log. That's harmless (the planted 22 violations and 3.3-day bottleneck don't move) — it actually makes the "live feed" story better.
- The chatbot uses the Cloudflare free-tier AI quota (10k neurons/day) — don't hammer it in rehearsal right before presenting.
- If the data ever needs a full reset, that's a terminal job back on the project machine (`scripts/seed.mjs --reset --base https://iie.ghwmelite.workers.dev`) with the production API key — ask whoever deployed it.

Option B — present from localhost (the original way)

You need: the project folder open, internet access (the AI features call live Cloudflare services even in local dev), and one terminal.

```
cd "C:/dev/IIE - Nii"
npm run dev           # starts the Worker + dashboard on http://localhost:8787
```

Then verify, in a second terminal:

```
curl http://localhost:8787/health      # expect: {"status":"ok"} (or similar 200 response)
curl http://localhost:8787/api/stats/overview
```

The overview should show roughly: **150 employees, ~33,300 events, ~4 flagged bottlenecks**. If it shows 0 events, the database is empty — run `npm run seed` (takes ~3 minutes) before the demo.

Open <http://localhost:8787> in your browser and click through all four tabs once to warm everything up. Keep this browser window ready. (Tabs are hash-routed — refreshing or sharing <http://localhost:8787#intelligence> lands directly on that tab.)

On-screen option: the dashboard has a built-in **Presenter Mode** — click "Presenter" in the header (top right) and a panel walks you through every step below: what to click, the exact line to say, and the number to expect. It navigates the tabs for you as you advance, and remembers your place if the page refreshes. Steps that point at something on screen also have a **"Take me there →"** button: it switches to the right tab, scrolls the exact panel into view, and flashes a highlight ring around it (on the "Open the assistant" step it opens the chat

widget for you). It never performs transactions — submitting leave, approving, and sending chat messages stay live clicks. This guide remains the full reference; Presenter Mode is the in-the-moment version of it.

If you ever need fresh data: `npm run seed -- --reset` wipes every table and re-seeds in one step (plain `npm run seed` is additive and duplicates events).

3. Know your data (the story behind the numbers)

The demo dataset simulates **6 months** of OHCS operations. Understanding it lets you answer "why does it show that?" confidently.

The organisation (real OHCS structure)

- **9 organisational units**, keyed by their real acronyms:
- 5 directorates: **RSIMD** (Research, Statistics & Information Management), **PBMED** (Planning, Budgeting, Monitoring & Evaluation), **CMD** (Career Management — where HR sits), **F&A** (Finance & Administration), **RTDD** (Recruitment, Training & Development)
- 4 units: **RCU** (Reform Coordinating Unit), **CSC** (Civil Service Council), **IAU** (Internal Audit Unit), **PR** (Public Relations Unit)
- **150 staff in total** (the full OHCS staff strength), following the civil-service grading structure: every unit has a **Director** (management), a **Deputy Director** and an **Assistant Director I** (middle management), plus **122 officers** of lower grades distributed across the units. One **HR officer** sits in CMD.
- Two deputy directors carry leave-administration designations: F&A's Deputy Director is the **admin officer** verifying standard leave; RTDD's Deputy Director is the **Schedule Officer** reviewing study leave.

The three event sources (~33,300 events total)

SOURCE	EVENTS	WHAT IT IS
ATTENDANCE	~31,700	RFID clock-in / clock-out taps, weekdays over 6 months
LEAVE_WORKFLOW	~1,200	Full leave-approval chains (256 cases)
CHATBOT	~400	Staff queries to the HR assistant

The prescribed leave workflows (policy)

Standard leave (annual, sick, maternity, casual) — administered at OHCS by F&A: `leave_submitted` → `supervisor_review` → `fa_verification` → `director_fa_approval` → `completed`

Study leave (OHCS and the entire Ghana Civil Service) — administered centrally by RTDD: `leave_submitted` → `supervisor_review` → `rtdd_review` → `director_rtdd_approval` → `completed`

- **Supervisor review** — the officer's middle-management supervisor in their own unit (Assistant Director / Deputy Director tier)
- **F&A verification / RTDD review** — the administering directorate's officer (F&A admin officer, or the RTDD Schedule Officer)
- **Director approval** — the Director of that directorate (**Director F&A**, or **Director RTDD** for study leave)

What the seed deliberately "planted" (this is what the system should find)

1. **A slow step:** F&A verification is intentionally slow — the transition `supervisor_review` → `fa_verification` has a **median of 3.3 days (P95: 5.7 days)**. The bottleneck detector should flag exactly this.
2. **Policy violations:** ~10% of leave cases **skip supervisor review entirely** (they jump straight to verification). The conformance checker should catch **all 22 of them**.

3. **Department outliers:** CSC has the worst punctuality (**13.8% late**); the Public Relations Unit has the slowest leave-approval cycle (**7.6 days**).

This planted ground truth is important: it means you can *prove* the mining works, because you know the right answers in advance.

4. Demo script — Act 1: Operations tab (3–4 min)

Narrative: *"This is the live operations view — what's happening across the office right now."*

Walk through top to bottom:

1. **System map** (top): shows the architecture — source subsystems (Attendance, Leave Workflow, Chatbot) feeding the Integration Engine, which feeds Process Intelligence. *"Every subsystem publishes events in one canonical format into a unified event log — that's the core design decision. New systems just start emitting events; nothing else changes."*
 2. **Stat cards:** 150 employees, open leave requests. *"These poll the API every 10 seconds — if an event arrives, the numbers move on their own."*
 3. **Live event feed** (left): the 25 most recent events, colour-coded by source (green = attendance, indigo = leave, amber = chatbot). *"This feed is pushed live over server-sent events — the moment any subsystem posts, it appears here. And every row is clickable."* **Click a row** — the case's full chronological event trace opens in a modal. *"Every case carries its complete audit trail."*
 4. **Attendance heatmap** (right): 30 days, green intensity = clock-in volume, red tint = days where >25% of staff were late. **Hover a red-tinted cell** to show the tooltip (e.g. "48 in, 9 late, 2 missing out"). *"You can spot bad Mondays at a glance."*
 5. **Leave pipeline:** counts of cases waiting at each stage — supervisor review, F&A verification, director F&A approval, RTDD review, director RTDD approval, completed, rejected, escalated. *"This is the state machine working — every leave request moves through one of the two prescribed chains, and the pipeline shows where work is piling up."*
-

5. Demo script — Act 2: Process Intelligence tab (5–6 min) — the centrepiece

Narrative: *"This is where the platform earns its name. Nobody drew this process map — the system discovered it from the event log."*

1. **The discovered process map** (top): - Point at the nodes and edges: *"Each node is an activity, each arrow is an observed hand-off with a case count and median transition time."* - **The two chains moment:** *"Look — the miner discovered BOTH prescribed chains on its own: standard leave flowing through F&A verification and Director F&A approval, and study leave flowing through RTDD review and Director RTDD approval. Nobody told it about the F&A/RTDD split — it read it out of the data."* - Point at the **red edges/pills:** *"Red means the transition is over its SLA threshold. The map overlays bottleneck analysis directly onto the discovered process."* - Mention the **rejected** branch (red-accented node): *"It also discovered the rejection paths on its own."* - Source switcher: click **ATTENDANCE** and **CHATBOT** briefly to show it mines all three sources, then return to LEAVE_WORKFLOW.
2. **Bottlenecks panel** (bottom left): - Read the top row: `supervisor_review -> fa_verification` — **median 3.3d, P95 5.7d**, flagged red. - *"The single slowest hand-off in the leave process is getting from the supervisor's desk to F&A verification — a median of over three days. That's a staffing or routing problem at one specific step, and the system localised it exactly."*
3. **Workflow variants** (bottom right): - *"256 leave cases collapse into a handful of distinct paths — the full F&A chain, the full RTDD study chain, the rejection variants... and these — `leave_submitted → fa_verification → ...` — where supervisor review never happened."*
4. **Conformance panel:** - *"The conformance checker compares every case against the prescribed model for its leave type. It found 22 cases that bypassed supervisor review — all type `skipped_step`. In a real deployment this is the audit list."* - Show the conformant-rate bar (~91%). - **Click a case id** — the full trace opens, showing the jump from `leave_submitted` straight to verification. *"Every finding is inspectable down to the raw events."*

If the supervisor asks "how do you know it's right?" — that's your cue: *"The demo data deliberately plants exactly these problems — a known slow step and 22 known violations — and the validation harness measures detection against that ground truth."* (Then show Act 6 if time permits.)

6. Demo script — Act 3: Decision Support tab (3–4 min)

Narrative: *"Mining results are for analysts. This page is for management — it turns the analysis into plain-English recommendations, ranked by severity."*

1. **Top insight banner** (dark card): the highest-severity finding — the slow `supervisor_review` → `fa_verification` step, with a concrete suggestion (backup approver / SLA alerts).
 2. **Department comparison** (the organisational-payoff moment): - *"Every unit, compared on two fixed scales: late-arrival rate and leave-approval cycle. Red marks anything more than 1.5× the office average."* - Point at **Civil Service Council — 13.8% late** (worst punctuality) and **Public Relations Unit — 7.6-day leave cycle** (slowest). - *"This is the kind of cross-directorate comparison OHCS currently can't produce without manual spreadsheets — here it's a by-product of the event log."*
 3. **Recommendation cards**: bottleneck alerts, the conformance finding ("22 cases bypassed supervisor review — enforce routing at submission"), and the variability note.
 4. **The export buttons** (top right): *"And this page leaves the building — **Download CSV** drops the recommendations and the department table straight into Excel, and **Print / Save as PDF** produces a clean one-pager for a director's memo."*
 5. Mention the roadmap line honestly: *"The recommendations are rule-generated today; an AI narrative layer over them is a planned next phase."*
-

7. Demo script — Act 4: the AI assistant (4–5 min)

Click **"Ask OHCS assistant"** (the gradient pill, bottom-right, on every page). The panel shows who's speaking via the **"Speaking as"** dropdown — *"the assistant knows who it's talking to; per-user sign-in arrives with the SSO phase."* The empty state offers **clickable suggestion chips** — tapping one asks it instantly (a handy shortcut mid-demo).

Narrative: *"The assistant is retrieval-augmented: it embeds the actual OHCS policy documents into a vector database and answers strictly from retrieved excerpts — it can't invent policy."*

Ask these in order (expected answers in brackets — the policy corpus guarantees them):

1. **"How many days of annual leave am I entitled to?"** → 30 working days. *Point at the indigo source chips under the reply — proof it's RAG over the real policy documents, not memorised.*
2. **"What is the grace period for morning clock-in?"** → 30 minutes (late after 8:30 a.m.).
3. **"Who approves study leave applications?"** → the Director, RTDD — after review by the RTDD Schedule Officer. *"That's the Study Leave Policy we just saw in the process map — the chatbot and the miner agree because both read the same reality."*
4. **"Do I need a medical certificate for two days of sick leave?"** → No — certificates only beyond 3 consecutive days.
5. **The personal-data moment: "How many days was I late this month?"** → answered from the *employee's own attendance records*, no LLM involved. *"This is the hybrid design: personal data queries go straight to the database through deterministic rules — the language model only handles policy language."*
6. **The transaction moment: "I'd like to request annual leave from 3 August 2026 to 7 August 2026."** - The assistant confirms: *"Done — your annual leave request (2026-08-03 to 2026-08-07) is submitted and now with your supervisor for review."* - **Then prove it's real:** flip to the **Operations tab** — the new `leave_submitted` event is already at the top of the live feed (pushed over SSE), and the pipeline's `supervisor review` count went up by one. - *"The chatbot didn't just talk — it executed a real workflow transaction against the state machine, and you watched the event land in the log."*

Safety point worth volunteering: *"Leave dates are only ever taken from the user's own message, never model-generated — the LLM can't invent dates for a transaction."*

8. Demo script — Act 5: My Leave (4–5 min) — the workflow made tangible

Narrative: *"The chatbot is one front door to the leave process. This is the other — a self-service portal where officers submit and track leave, and approvers act on it. Same state machine underneath."*

1. **Submit:** with "Acting as" set to any officer, pick *annual*, choose dates, **Submit**. The request appears under **My requests**, *pending · waiting at supervisor review*.
 2. **Approve the full chain in the Approver inbox** (right side): - **Supervisor review** — pick the officer's unit Assistant Director I from the dropdown (*"the picker only offers officers whose role and unit allow them to act — the state machine would reject anyone else"*) → **Approve**. The request moves to **F&A verification**. - **F&A verification** — the picker now only offers the F&A admin officer (the F&A Deputy Director) → **Approve**. Moves to **director F&A approval**. - **Director F&A approval** — only Director F&A is offered → **Approve**. The request lands in **completed**. - Flip back to **Operations**: the whole chain just streamed through the live feed.
 3. **Show the routing rule:** submit a **study** request, approve at supervisor review — and point at the inbox: *"Study leave doesn't go to F&A. It routes to RTDD — the Schedule Officer reviews, and only Director RTDD can approve. That's the real Civil Service rule, enforced by the same engine the miner audits against."*
 4. **Track:** expand a request under My requests — the step-by-step timeline with actor and timestamp. *"Every officer can see exactly where their request is and whose desk it's on."*
-

9. Demo script — Act 6 (optional, for technical supervisors): proof it works (3 min)

This act is the only one that needs a terminal, and you can also just talk through the results table below instead — the harness output is reproducible on request.

Against the **live site** (from the project folder, no dev server needed):

```
# Windows (Git Bash): the production API key is required for the latency test
IIE_API_KEY=<production-key> node scripts/validate.mjs --base https://iie.ghwmelite.workers.dev
# or skip the AI metric to save free-tier quota:
IIE_API_KEY=<production-key> node scripts/validate.mjs --base https://iie.ghwmelite.workers.dev --no-a
```

Or against **localhost**, with the dev server running (`npm run dev`):

```
npm run validate
```

This measures the project's six PRD success metrics live, against the planted ground truth. All six pass:

METRIC	TARGET	TYPICAL RESULT
Event capture latency	< 500 ms	p50 ~8 ms
Workflow variants discovered	≥ 3	9
Bottleneck detection	F&A-verification transition flagged	<code>supervisor_review</code> → <code>fa_verification</code> , median 3.3d
Conformance detection rate	> 80% of known violations	22/22 = 100%, 0 false positives
Dashboard load time	< 2 s	< 100 ms
Chatbot policy resolution	> 60%	5/5 = 100%

If they want more: `npm test` runs the 32-test suite (ingestion, the leave state machine — including the study-leave RTDD routing and F&A scoping, DFG miner, conformance checker, SSE stream, full mining pipeline) inside the real Workers runtime.

You can also show raw API output in the browser — e.g.

`https://iie.ghwmelite.workers.dev/api/intelligence/recommendations` (or `http://localhost:8787/...` when presenting locally) — *"everything the dashboard shows is a JSON API; any other system can consume it."*

10. Questions supervisors commonly ask — and strong answers

"Where does the data come from in a real deployment?" The subsystems (RFID attendance controllers, the leave system, the chatbot) POST canonical events to `/api/events` — authenticated with an API key (timing-safe comparison). The demo uses a seeded simulation of 6 months because real subsystems aren't integrated yet.

"Is the process map hand-drawn?" No — it's a directly-follows graph mined from event sequences (case IDs + timestamps). Layout is automatic (BFS layering). If the real process changes, the map changes on the next mining run. The F&A/RTDD split in the leave process was *discovered*, not configured.

"How are the two leave chains enforced?" One state machine (`src/lib/workflow.ts`): every request needs its supervisor's review first; standard types then route to F&A (admin officer verifies, Director F&A approves), study leave to RTDD (Schedule Officer reviews, Director RTDD approves). Role *and* directorate checks — an RTDD director can't approve annual leave, an F&A officer can't touch study leave.

"How current is the analysis?" A cron trigger runs mining every 6 hours automatically; the dashboard polls every 30 seconds, and the event feed is pushed live over server-sent events. Mining can also be triggered on demand (`POST /api/intelligence/run`).

"What AI models does it use?" Intent classification: Llama 3.2 3B. Policy answers: Llama 3.3 70B, grounded in BGE-base embeddings over Vectorize. Models are config values in `wrangler.jsonc`, swappable. Personal-data queries deliberately bypass the LLM entirely.

"What about security / who can see what?" Machine-to-machine ingestion endpoints require an API key. Per-user identity (Cloudflare Access / SSO) is a planned next phase — the "Acting as" pickers are explicit placeholders for that.

"Can the thresholds be tuned without redeploying?" Yes — bottleneck SLA thresholds live in a KV config namespace (`bottleneck_thresholds_ms`), read at mining time with code defaults as fallback. Changing them is one `wrangler kv key put` command.

"Why Cloudflare Workers?" Serverless at the edge: no server to manage, D1 (SQLite) and Vectorize colocated with the compute, free tier covers the whole demo, and one `wrangler deploy` publishes it. (The repo includes an architecture diagram — `iie_cloudflare_architecture.svg` — if they want the visual.)

"What's the conformance checker actually checking against?" The prescribed leave workflows — the same state machine that enforces live transitions. One source of truth: the model that approves requests is the model that audits them.

"What are the limitations / next phases?" Be honest — it lands well: SSO identity, queue fanout for ingestion bursts, AI narrative over recommendations, and moving the prescribed conformance model into config.

11. Troubleshooting (if something goes wrong mid-demo)

Presenting from the **live site**, the likely symptoms are just these two:

SYMPTOM	FIX
Dashboard blank after clicking a tab	Hard refresh (Ctrl+F5) — you may have a stale cached bundle.
Chatbot says "Something went wrong"	Wait a few seconds and retry — free-tier AI rate limits (10k neurons/day). If it's out for the day, demo the remaining tabs; everything else is unaffected.

The rows below apply when presenting from **localhost** (Option B):

SYMPTOM	FIX
Dashboard blank after clicking a tab	Hard refresh (Ctrl+F5) — you may have a stale cached bundle.
Panels say "Couldn't load ..."	The API is unreachable — check <code>npm run dev</code> is still running; the dashboard retries on its own.
Chatbot says "Something went wrong"	Check internet — Workers AI/Vectorize are live remote services even in local dev. Wait a few seconds and retry (free-tier rate limits).
Stat cards show 0 employees / 0 events	DB is empty: run <code>npm run seed</code> (~3 min), then refresh.
Numbers look doubled (e.g. 66k+ events)	Seed was re-run without <code>--reset</code> . Cosmetic only — or wipe cleanly with <code>npm run seed -- --reset</code> before the demo.
Port 8787 already in use	Another <code>wrangler dev</code> is running — close the old terminal, or just use the already-running one.
<code>npm run validate</code> fails on chatbot metric only	AI free-tier quota (10k neurons/day) — demo the chatbot live instead; it shares the same path.

12. One-minute recap (your closing lines)

"To summarise: one unified event log from every subsystem; automatic discovery of the real process — both leave chains, F&A and RTDD, found by the miner itself; bottleneck and conformance analysis validated at 100% against known ground truth; management-ready recommendations with department comparisons across all nine OHCS units, exportable to CSV and PDF; self-service leave that enforces the real approval rules; and an AI assistant that answers policy from the actual documents and executes real transactions. Everything you saw is running on serverless infrastructure that deploys with a single command."

Generated July 2026 against the seeded dataset (seed 42: 150 staff, 9 units, 33,326 events, 6 months). Numbers quoted are live from the local instance at the time of writing.